Aula 9: O Contrato de Comportamento - Interfaces



Projeto: FiapRide (Sistema de Mobilidade Urbana) + Seu Projeto Pessoal

Foco: Interfaces, Contratos e Múltiplos Tipos

🎯 Objetivo da Aula

Até agora, aprendemos que uma classe pode ter apenas UMA mãe (herança simples). Um `Carro` herda de `Veiculo`. Um `Cachorro` herda de `Pet`. Mas e se um `CarroEletrico` pudesse ser tratado como "um Veículo" E TAMBÉM como "algo que pode ser recarregado"?

Pense bem: Um `CarroEletrico` É UM Veículo (herança), mas ele TAMBÉM É recarregável. E um `Celular`? Ele NÃO É um Veículo (está em outra hierarquia), mas TAMBÉM é recarregável!

Hoje você vai aprender sobre **Interfaces**. Vamos criar contratos de comportamento que permitem que classes de hierarquias diferentes compartilhem capacidades, sem ficarem presas em uma única "árvore genealógica".

Conceitos Chave:

- **Interface:** Um contrato que diz "O QUE" uma classe deve fazer, não "COMO" fazer.
- **Contrato de Comportamento:** Define assinaturas de métodos que TODA classe que assina deve implementar.
- **Múltiplos Tipos:** Uma classe pode HERDAR de uma mãe E IMPLEMENTAR várias interfaces.

📖 A Teoria: O Menu do Restaurante vs. A Cozinha

Pense em um restaurante.

- **O Menu (Interface):** Lista o que é oferecido (tem comida, tem bebida, tem sobremesa). Não diz COMO a comida é feita, apenas que ELA EXISTE. Qualquer restaurante que tenha esse menu deve OFERCER esses itens.
- **A Cozinha (Classe Concreta):** É onde a comida é DE FATO preparada. Cada restaurante implementa o menu do jeito que quiser (cozinha italiana, japonesa, brasileira).

A Diferença Crucial:

- **Classe Abstrata:** É um "molde parcial". Define comportamentos comuns e deixa alguns para as filhas.
- **Interface:** É um "contrato puro". Define APENAS assinaturas (sem código). Diz "vai ter esse método", mas não implementa nada.

Quando usar Interface vs. Classe Abstrata:

- **Use INTERFACE se:** Você quer definir um COMPORTAMENTO que classes DIFERENTES (sem relação de herança) podem ter.
- **Use CLASSE ABSTRATA se:** Você quer compartilhar CÓDIGO entre classes relacionadas (mesma hierarquia).

No FiapRide:

- `Veiculo` é **CLASSE ABSTRATA** porque `Carro` e `Moto` compartilham código (placa, modelo).
- `Recarregavel` é **INTERFACE** porque `CarroEletrico` e `Celular` não são parentes, mas AMBOS recarregam!

🛠️ Parte 1: Modelagem Visual (Astah)

Vamos aprender a Importar o código para o Astah



Método 1: Pelo Menu Tradicional

1. **Abra o Astah** e crie um novo projeto (ou abra o projeto onde deseja colocar os diagramas).
2. No menu superior, clique em **Tool** (Ferramentas).
3. Vá em **Java** e depois clique em **Import Java...** (Importar Java...).
4. Uma nova janela será aberta. Aqui você tem duas opções:
   - Clique em **Add...** (Adicionar) para selecionar arquivos `.java` específicos.

- Clique em **Add Dir...** (Adicionar Diretório) para selecionar uma pasta inteira (ele vai buscar todos os arquivos Java dentro daquela pasta e subpastas).

5. Na parte inferior dessa mesma janela, selecione o **Encoding** (Codificação) correto do seu projeto. É altamente recomendável escolher **UTF-8** para evitar que comentários ou strings com acentos fiquem com caracteres estranhos.
6. Clique em **OK**. O Astah fará a leitura do seu código.

Como visualizar no diagrama:O código importado **não** aparece automaticamente como um desenho na tela. Ele é carregado para a estrutura do projeto.

7. Vá para o painel esquerdo, chamado **Structure Tree** (Árvore de Estrutura).

8. Crie ou abra um **Class Diagram** (Diagrama de Classes).

9. Encontre as classes ou pacotes que você acabou de importar na *Structure Tree*, clique neles e **arraste-os** para a área em branco do seu diagrama de classes. O Astah montará as caixas com os atributos, métodos e as relações automaticamente.

Método 2: O Jeito Rápido (Arrastar e Soltar)

Se você já estiver com um diagrama aberto, existe um atalho muito prático:

1. Abra o Astah e crie um **Class Diagram** (Diagrama de Classes) em branco.
2. Abra o seu explorador de arquivos (Windows Explorer, Finder, etc.) e localize a pasta ou os arquivos `.java` do seu projeto.
3. Simplesmente **selecione, arraste e solte** os arquivos ou a pasta de código de dentro do seu computador diretamente para a tela de desenho do Astah.
4. Ele fará a importação automaticamente e já colocará as classes desenhadas na tela.

💻 Parte 2: O Desafio FiapRide (Mão na Massa)

Passo 1: Criando a Interface Recarregável

Vamos criar o contrato de recarregamento.

```
package br.com.fiapride.model;

// 'interface' em vez de 'class'
// Todo método é PUBLICO e ABSTRATO por padrão (não precisa escrever)
public interface Recarregavel {

    // Método sem corpo (apenas assinatura)
    // Define QUEM implementa recarregar() DEVE ter este método
    void recarregar(int percentual);

    // Constantes são permitidas em interfaces
    int CARGA_MAXIMA = 100;
    int CARGA_MINIMA = 0;

    // Observação: Em interfaces Java:
    // - Métodos são public abstract por padrão
    // - Atributos são public static final por padrio (constantes)
```

```
}
```

🔍 Por que Constantes de Interface são `public static final`?

Você notou que declaramos:

```
int CARGA_MAXIMA = 100;  // Sem escrever "public static final"
```

Mas **NUNCA** escrevemos:

```
public static final int CARGA_MAXIMA = 100;  // Redundante!
```

📝 A Regra de Ouro

> **Atributos de interface são IMPLICITAMENTE `public static final`**
>
> **Escrever `public static final` em interface é redundante, mas funciona.**

🧐 Por que `public static final`?

Modificador	O que faz	É implícito em interface?
`public`	Pode ser acessado por qualquer classe	✅ **SIM**
`static`	Pertence à interface, não ao objeto	✅ **SIM**
`final`	Não pode ser alterado (constante)	✅ **SIM**

Explicação:

- `public` - Constante deve ser acessível por todos que implementam a interface
- `static` - Constante é da interface, não de cada objeto (compartilhada)
- `final` - Constante NÃO pode ser alterada (é fixa, imutável)

📊 Comparação: O que compila vs. o que é correto

Combinação	Compila?	Correto?	Explicação
`int CARGA_MAXIMA = 100;`	✅ Sim	✅ **SIM**	Forma mais limpa (implícito)
`public static final int CARGA = 100;`	✅ Sim	⚠️ Redundante	Funciona, mas é repetitivo
`public int CARGA = 100;`	❌ **NÃO**	❌ Não	**IMPOSSÍVEL!** Deve ser `final`
`private int CARGA = 100;`	❌ **NÃO**	❌ Não	**IMPOSSÍVEL!** Constantes de interface são `public`
`static int CARGA = 100;`	❌ **NÃO**	❌ Não	**IMPOSSÍVEL!** Deve ser `public final`

🎯 Formas de Acessar Constantes

Forma 1: Através da Interface ✅ (RECOMENDADO)

```
int max = Recarregavel.CARGA_MAXIMA;  // ✅ PADRÃO
```

Forma 2: Através da Classe que Implementa ⚠️ (Funciona, mas não é padrão)

```
public class CarroEletrico implements Recarregavel {
    public void teste() {
        int max = this.CARGA_MAXIMA;  // ⚠️ Funciona (herdado)
        int max2 = CARGA_MAXIMA;      // ⚠️ Funciona (sem this)
    }
}
```

📋 Boas Práticas: Nomeação de Constantes

Regra	Exemplo	❌ Errado
TODA MAIÚSCULA	`CARGA_MAXIMA`	`carga_maxima`
Palavras separadas por `_`	`TAXA_DESCONTO`	`TAXADESCONTO`
Significado claro	`CAPACIDADE_MAXIMA`	`MAX`
Sem números mágicos	`LIMITE_CARROS = 10`	`10` (no código)

💡 Analogia do Mundo Real

Constante de Interface = Placa na Parede

```
┌──────────────────────────┐
|     RESTAURANTE          |
|                          |
|  Horário: 11h às 23h     |  ← Todos veem (public)
|  (Não pode mudar)        |  ← Fixo na parede (final)
|  (Igual para todos)      |  ← Compartilhado (static)
└──────────────────────────┘

- Qualquer um pode LER (public)
- Ninguém pode ALTERAR (final)
- É o MESMO horário para todos (static)
```

Passo 2: CarroEletrico Implementando a Interface

Agora criamos um carro elétrico que IMPLEMENTA o contrato.

```
package br.com.fiapride.model;

// 'implements' = assina o contrato
public class CarroEletrico extends Veiculo implements Recarregavel {

    private int nivelBateria;

    public CarroEletrico(String placa, String modelo) {
        super(placa, modelo);
    }
```

```
    // OBRIGATÓRIO: Implementar método da interface
    @Override
    public void recarregar(int percentual) {
        if (percentual < Recarregavel.CARGA_MINIMA || percentual > Recarregavel.CARGA_MAXIMA) {
            System.out.println("Erro: Percentual inválido!");
            return;
        }

        System.out.println("Carro " + this.getModelo() + " recarregado para " + percentual + "%");
        this.setNivelBateria(percentual);

    }



    @Override
    public String calcularAutonomia() {
        double kmRestante = this.getNivelBateria() * 5.0;
        return "Autonomia: " + kmRestante + " km.";
    }

    public int getNivelBateria() {
        return nivelBateria;
    }

    public void setNivelBateria(int nivelBateria) {
        this.nivelBateria = nivelBateria;
    }
}
```

Passo 3: Celular Implementando a Interface

Agora algo fora da hierarquia Veículo!

```
package br.com.fiapride.model;

// Celular NÃO herda de Veiculo, mas TAMBÉM é Recarregavel
public class Celular implements Recarregavel {

    private String marca;
    private int nivelBateria; // 0-100

    public Celular(String marca) {
        this.marca = marca;
        this.nivelBateria = 0;
    }

    @Override
    public void recarregar(int percentual) {
```

```
        if (percentual < Recarregavel.CARGA_MINIMA || percentual > Recarregavel.CARGA_MAXIMA) {
            System.out.println("Erro: Percentual inválido!");
            return;
        }

        this.nivelBateria = percentual;
        System.out.println("Celular " + this.marca + " carregado para " + percentual + "%");
    }

    public int getNivelBateria() {
        return this.nivelBateria;
    }
}
```

Passo 4: O Teste Polimórfico (SistemaPrincipal)

Vamos provar que podemos tratar objetos diferentes como `Recarregavel`.

```
package br.com.fiapride.main;

import br.com.fiapride.model.CarroEletrico;
import br.com.fiapride.model.Celular;
import br.com.fiapride.model.Recarregavel;
import br.com.fiapride.model.Veiculo;

public class SistemaPrincipal {
    public static void main(String[] args) {

        System.out.println("=== FIAPRIDE: Teste de Interfaces ===");
        System.out.println();

        // Criamos objetos de classes DIFERENTES
        CarroEletrico tesla = new CarroEletrico("TES-9999", "Tesla Model 3");
        Celular iphone = new Celular("iPhone 15");

        // POLIMORFISMO DE INTERFACE:
        // Ambos podem ser tratados como 'Recarregavel'
        Recarregavel[] recarregaveis = new Recarregavel[] {
            tesla,
            iphone
        };

        System.out.println("=== Recarregando Tudo ===");
        for (Recarregavel r : recarregaveis) {
            r.recarregar(85); // Mesmo método, comportamentos diferentes
            System.out.println("---");
        }

        System.out.println();
        System.out.println("=== Múltiplos Tipos ===");
```

```
        // CarroEletrico É UM Veiculo E TAMBÉM É UM Recarregavel
        System.out.println("Tesla é um Veiculo? " + (tesla instanceof Veiculo)); // true
        System.out.println("Tesla é Recarregavel? " + (tesla instanceof Recarregavel)); // true

        // Celular NÃO é Veiculo, MAS é Recarregavel
        System.out.println("Celular é um Veiculo? " + (Veiculo.class.isInstance(iphone))); // false
        System.out.println("Celular é Recarregavel? " + (iphone instanceof Recarregavel)); // true

        System.out.println();
        System.out.println("=== Autonomia do Carro Elétrico ===");
        System.out.println(tesla.calcularAutonomia());

        System.out.println();
        System.out.println("=== Bateria do Celular ===");
        System.out.println(iphone.getNivelBateria());
    }
}
```

Saída esperada no console:

```
=== FIAPRIDE: Teste de Interfaces ===

=== Recarregando Tudo ===
Carro Tesla Model 3 recarregado para 85%
---
Celular iPhone 15 carregado para 85%
---

=== Múltiplos Tipos ===
Tesla é um Veiculo? true
Tesla é Recarregavel? true
Celular é um Veiculo? false
Celular é Recarregavel? true

=== Autonomia do Carro Elétrico ===
Autonomia: 425.0 km.

=== Bateria do Celular ===
85
```

🧐 Por que Interfaces Permitem "Herança Múltipla"?

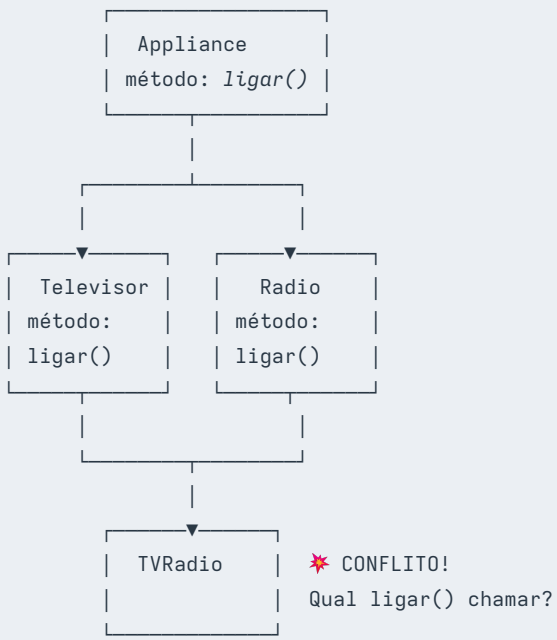
> 🎯 **O Grande Paradoxo:**
>
> Java permite **APENAS 1 herança** (uma mãe), mas **VÁRIAS interfaces** (vários contratos). Por quê?

O Problema da Herança Múltipla

Imagine se Java permitisse herdar de DUAS classes ao mesmo tempo:

```
// ❌ CENÁRIO HIPOTÉTICO (Java NÃO permite!)
public class CarroEletrico extends Veiculo, Bateria {
    // Qual "ligar()" chamar? O da mãe Veiculo ou da mãe Bateria?
}
```

💥 O Conflito do "Diamond Problem"



Problema: Se `Televisor` e `Radio` implementam `ligar()` de formas diferentes, `TVRadio` não sabe qual usar!

✅ A Solução das Interfaces

Interfaces **NÃO têm código**, apenas assinaturas. Sem código = Sem conflito!

```
// ✅ Java PERMITE várias interfaces!
public class CarroEletrico extends Veiculo implements Recarregavel, GPS, Autonomia {
    // Veiculo TEM código (herdado)
    // Recarregavel, GPS, Autonomia = apenas contratos (sem conflito)
}
```

📊 Comparação: Herança Múltipla de Classes vs. Interfaces

Aspecto	Herança Múltipla de Classes	Múltiplas Interfaces
Java permite?	❌ NÃO	✅ SIM
Risco de conflito	⚠️ ALTO (mesmo método em mães diferentes)	✅ NENHUM (apenas assinaturas)
Tem código?	✅ Sim (implementações concretas)	❌ Não (apenas contratos)

Aspecto	Herança Múltipla de Classes	Múltiplas Interfaces
Problema Diamond	⚠️ Pode ocorrer	✅ Impossível (sem código)
Complexidade	❌ Alta	✅ Baixa
Exemplo	`extends Veiculo, Bateria` ❌	`implements Recarregavel, GPS` ✅

🧪 Experimento Prático: Várias Interfaces

```
// Interface 1: Comportamento de recarregar
public interface Recarregavel {
     void recarregar(int percentual);

}

// Interface 2: Comportamento de GPS
public interface GPS {
    void calcularRota(String destino);
}

// Interface 3: Comportamento de autonomia
public interface Autonomia {
    int getAutonomiaRestante();
}

// ✅ CarroEletrico implementando TODAS!
public class CarroEletrico extends Veiculo implements Recarregavel, GPS, Autonomia {

    @Override
    public void recarregar(int percentual) {
        System.out.println("Recarregando...");
    }

    @Override
    public void calcularRota(String destino) {
        System.out.println("Rota para: " + destino);
    }

    @Override
    public int getAutonomiaRestante() {
        return 350;
    }
}
```

Por que funciona?

- Cada interface define **MÉTODOS DIFERENTES**
- Mesmo que duas interfaces tivessem métodos com o MESMO nome, não há conflito porque:
- Você implementa **UMA vez** e satisfaz AMBAS

- Não há "código duplicado" de mães diferentes

🎯 Regra de Ouro

> **Herança múltipla de classes = CONFLITO 💥**
>
> **Múltiplas interfaces = CONTRATOS ✅**

- **Classe mãe:** TEM código → Pode conflitar com outra mãe
- **Interface:** TEM contrato (assinatura) → Sem conflito possível

💡 Analogia do Mundo Real

Herança Múltipla de Classes (PROBLEMA):

```
Você tem 2 mães biológicas.
Mãe 1 diz: "Fale português"
Mãe 2 diz: "Fale inglês"
💥 CONFLITO: Qual língua falar?
```

Múltiplas Interfaces (SOLUÇÃO):

```
Você tem 1 mãe biológica + 2 contratos de trabalho.
Mãe: Você TEM olhos azuis (herdado)
Contrato 1: Você PRECISA saber programar
Contrato 2: Você PRECISA saber liderar
✅ SEM CONFLITO: Cada contrato é independente!
```

📋 Por que Java Tomou Essa Decisão?

Motivo	Explicação
Simplicidade	Interfaces não têm código, então não há "máxima" para resolver
Segurança	Sem conflito de implementação, o código é mais previsível
Flexibilidade	Você pode ter múltiplos "tipos" sem dor de cabeça
Manutenibilidade	Mais fácil entender contratos do que heranças complexas

🔬 Teste Rápido

Qual destos compila em Java?

```java
// 1) Herança múltipla de classes
class A extends B, C { }

// 2) Múltiplas interfaces
class D implements E, F, G { }
```

```
// 3) Herança + Interface
class H extends I implements J, K { }

// 4) Interface estendendo interface
interface L implements M, N { }
```

Respostas:

1. ❌ Não compila (herança múltipla de classes)
2. ✅ Compila (múltiplas interfaces)
3. ✅ Compila (herança + interfaces)
4. ✅ Compila (interfaces podem estender múltiplas interfaces)

🧠 Checklist: Clean Code (Interfaces)

- ☐ **Nome descritivo:** Use adjetivos que indicam CAPACIDADE (ex: `Recarregavel`, `Voavel`, `Pagavel`).
- ☐ **Convenção "able":** Em inglês, interfaces terminam em "-able". Em PT, use adjetivos.
- ☐ **Contrato claro:** A interface deve definir um COMPORTAMENTO coeso (não misture coisas aleatórias).
- ☐ **Poucos métodos:** Interfaces com MUITOS métodos são difíceis de implementar (Interface Segregation Principle).
- ☐ **Documente:** Explique no JavaDoc O QUE esperar de quem implementa a interface.

⚡ Microdesafio Pessoal: Crie suas Interfaces

Volte para a classe do **seu** projeto pessoal.

Sua Missão:

1. Pense em um COMPORTAMENTO que objetos DIFERENTES podem ter.
   - *Ex:* `Arrastavel` *(pode ser arrastado) - implementado por* `Mesa`, `Cadeira`, `Caixa`.
   - *Ex:* `Vendavel` *(pode ser vendido) - implementado por* `Produto`, `Servico`, `Assinatura`.
   - *Ex:* `Notificavel` *(pode enviar notificações) - implementado por* `Email`, `SMS`, `PushNotification`.
2. Crie uma interface com 1-2 métodos.
3. Faça 2-3 classes (podem ser de hierarquias DIFERENTES) implementarem essa interface.
4. No `main`, crie um array da interface e chame os métodos polimórficamente.

🧠 Pergunta de Reflexão

"Por que Java permite herança simples (apenas uma mãe), mas múltipla implementação de interfaces (vários contratos)?

Pense: Se `CarroEletrico` pudesse herdar de `Veiculo` E de `Bateria` ao mesmo tempo (herança múltipla), o que aconteceria se AMBAS as mães tivessem um método chamado `ligar()`?

Como as interfaces resolvem esse problema? "

📤 Finalizando (Versionamento)

> 💡 **Você aprendeu a fazer commits na Aula 1.**
>
> Execute os comandos abaixo para salvar seu progresso:

```
git add .
git commit -m "feat: implementa interface"
git push origin main
```

Bons estudos!

